AIDTM STUDENT SUPPORT AGENT

Technical Project Documentation | Agentic AI Engineering – Assignment 1

Abhi Satapara – BDA - 20251001 | Batch 2025–27

1. Project Overview

The AIDTM Student Support Agent is a chat-style student assistance application built to demonstrate agentic AI concepts. It accepts student questions and can answer using a local knowledge base, retrieve relevant policy context, call structured tools, and use remembered student preferences or programme information.

The implementation uses a Streamlit interface, a Groq-hosted language model (openai/gpt-oss-120b), Python tool functions, JSON-backed demonstration data, a lightweight BM25 retrieval component, and LangSmith tracing for internal observability.

2. Assignment Objectives and Coverage

Assignment requirement	Implementation / evidence
Working agent loop	The model receives the user question and available context, decides whether a tool is needed, uses tool output when called, and produces a final response.
At least two tools	Implemented tools include fee-status lookup, payment-policy retrieval, support-request creation, and student-memory operations.
Persistent memory	A student can explicitly ask the agent to remember durable information such as programme or response preferences, then retrieve it in a later interaction.
Policy retrieval	The agent searches local knowledge/policy JSON content and should avoid treating unrelated fee information as a payment exception.
Context selection	The application supplies question-relevant knowledge or memory rather than relying on the whole knowledge collection for every answer.
Missing information / errors	Unsupported questions are answered with an uncertainty statement instead of fabricated eligibility criteria or deadlines.
Tracing and measurement	LangSmith traces and local structured traces record tools, latency, prompt tokens and completion tokens where reported.

3. System Architecture

Student → Streamlit chat interface → agent orchestration / Groq chat model → (a) direct response, (b) tool call, (c) BM25 knowledge retrieval, or (d) persistent-memory operation → final student-facing response. LangSmith receives execution traces for monitoring; local JSON files persist demo records, chat history, memory and support requests.

Main components:

- app.py – Streamlit interface, session/chat handling, model interaction and tool orchestration.
- support_tools.py – structured student-support tools such as fee lookup, policy retrieval, ticket creation and memory operations.

- retrieval.py – BM25-style keyword retrieval over local knowledge JSON documents.
- knowledge_ingestion.py – prepares or refreshes local knowledge JSON content.
- data/knowledge/ – AIDTM information and policy-oriented knowledge files.
- data/students.json – fictional sample student records for fee-status demonstrations.
- data/student_memory.json – persistent non-sensitive student facts.
- data/support_requests.json – locally stored support-request demonstration records.
- data/chat_history.json – locally persisted conversation history.
- data/agent_traces.json – local structured trace log containing tool, latency and token fields.

4. Agentic Workflow

1. Receive the student's message and the active student/session context.
2. Inspect the request and decide whether it can be answered directly or requires retrieval, memory, or a structured tool.
3. If required, call the selected tool with the necessary arguments.
4. Use the returned tool result or retrieved evidence to formulate the answer.
5. If evidence is missing or a tool fails, communicate the limitation and avoid inventing institutional policy.
6. Record execution metadata for evaluation and tracing.

5. Tools and Memory

Tool	Purpose
check_student_fee_status	Reads a fictional sample student record and returns programme/fee totals, paid amount, pending amount and status.
retrieve_payment_policy	Searches local knowledge for an explicit payment exception; a fee schedule alone is not treated as permission to pay late.
create_support_request	Creates a local demonstration ticket and returns its ticket ID, category and status. It does not contact AIDTM.
remember_student_fact / get_student_memory	Stores and retrieves durable, non-sensitive facts such as programme or answer-style preference. The user is expected to explicitly request remembering.

The screenshots demonstrate fee-status retrieval, payment-policy retrieval, support-ticket creation, memory storage/retrieval, an unsupported scholarship question and a general educational question.

6. Knowledge Retrieval and Grounding

The project uses local JSON knowledge files covering AIDTM overview, admissions, curriculum, faculty, fees, placements, programmes and student facilities. The retrieval module uses BM25 keyword search. The application should pass only relevant retrieved content to the model. If the requested fact is absent, the agent should state that it does not have the specific information.

Important policy limitation: the payment-next-working-day response shown in the test screenshot is based on a sample/illustrative payment-failure policy in the project. It must not be represented as an officially approved AIDTM policy unless separately verified by the institute.

7. Interface and Demonstrated Scenarios

- Fee status: the chatbot displayed a PGDM Big Data Analytics sample record with total fee ₹50,00,000, paid ₹27,00,000, pending ₹23,00,000 and status Partially Paid (as displayed in the screenshot; verify figures against the demo record before any reuse).
- Payment portal failure: the chatbot described the sample next-working-day process and evidence to retain, and advised not to retry if money had been debited until transaction verification.
- Support request: the chatbot confirmed ticket AIDTM-821EB5CB, category Fee Payment Issue, status Open.
- Persistent memory: the agent stored/recalled programme PGDM in Big Data Analytics and a preference for simple explanations.
- Unsupported scholarship information: the chatbot did not invent 100% scholarship eligibility criteria or an application deadline and directed the student to admissions for current confirmation.
- General knowledge: the chatbot explained AI, ML and DL with simple examples.

8. Observability with LangSmith

LangSmith tracing is enabled for the AIDTM Student Support Agent project. The supplied trace screenshots show the model identifier openai/gpt-oss-120b and execution fields including latency_seconds, token_usage (prompt_tokens and completion_tokens), tools_used, and tool success indicators. The local agent_traces.json file also stores timestamp, question, tools, latency_seconds, prompt_tokens and completion_tokens.

LangSmith is used for developer-side evaluation and debugging. Internal trace details and token counts are not intended for display in the student chat interface.

9. Evaluation Method and Results Summary

The evaluation workbook supplied with this documentation contains test cases derived from the visible chatbot screenshots and LangSmith trace evidence. Quantitative values are included only where legible in the screenshots. Some screenshots show the chatbot response without a readable matching trace, so those metric cells are marked Not captured / Not legible. The workbook distinguishes observed behaviour from inferred pass/fail assessment.

Visible trace readings transcribed from the supplied screenshots include: fee-status tool run – 1.914 s, 3,021 prompt tokens, 144 completion tokens; payment-policy retrieval – 2.421 s, 4,304 prompt tokens, 272 completion tokens; memory retrieval trace – approximately 2.287 s, 1,115 prompt tokens, 127 completion tokens; general AI/ML/DL answer – 2.719 s, 3,253 prompt tokens, 204 completion tokens. These are individual sample runs, not benchmark averages.

A scholarship trace is visible, but its token values are too small/unclear in the supplied screenshot to transcribe confidently. No unsupported numerical values are substituted.

10. Limitations, Privacy and Responsible Use

- Student records are fictional demonstration data. The student-ID selector is not institutional authentication.
- Support requests are written to a local JSON file; they are not submitted to an institute ticketing system.
- The payment-failure policy content is illustrative and must be verified before any real-world use.
- Local JSON storage is suitable for a classroom prototype, not production student records.
- API credentials must be stored in a private .env file and excluded from submission/version control.
- Knowledge content can become outdated; important fee, admission and scholarship information must be confirmed with AIDTM.

- Model answers can vary between runs; the evaluation results are observations from the submitted test evidence, not a guarantee of universal accuracy.

11. Conclusion

The project demonstrates a working student-support agent that combines tool use, retrieval, persistent memory, context-aware answering, uncertainty handling and tracing. The screenshots provide evidence of several end-to-end scenarios, while the evaluation workbook records the observed outcomes and available performance measurements. The implementation is a classroom prototype and would require secure authentication, protected databases, verified institutional policies and operational ticket integration before deployment.